UNIVERSITÀ DEGLI STUDI DI UDINE
hic sunt futura

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

PROGETTO DI LINGUAGGI E COMPILATORI

Parte 1

Francesco Bortuzzo **157430**
Leonardo Danelutti **158786**
Lorenzo Della Giustina **158854**

Anno Accademico 2024/2025

Indice

1. Grammatica labelled-BNFC 1
2. Import 2
   2.a. Path 2
   2.b. Import ricorsivi e multipli 2
3. DataStructure 4
   3.a. Creazione della struttura dati 4
   3.b. Risoluzione riferimenti 5
   3.c. Metodi 5
      3.c.I. Risoluzione del valore di un field 5
      3.c.II. Aggiunta 5
      3.c.III. Cancellazione 5
4. Pretty Printer 6
   4.a. Struttura principale 6
   4.b. Commenti 6
5. Demo 8

1. Grammatica labelled-BNFC

Si è scelto di implementare il progetto in C mediante BNFC. La grammatica con cui è stato generato il codice da BNFC è la seguente:

<XML> → ε | <Import> <XML> | <Section> <XMLS>
<XMLS> → ε | <Section> <XMLS>
<Import> → String
<Section> → "<" "section" "name" "=" <Ident> ">" <SBody> "<\" "section" ">"
<SBody> → ε | <Inherit> <SBody> | <Field> <SBodyF> (1)
<SBodyF> → ε | <Field> <SBodyF>
<Inherit> → "<" "inherit" ">" <Ident> "<\" "inherit" ">"
<Field> → "<" "field" "name" "=" <Ident> ">" <Value> "<\" "field" ">"
<Value> → <Integer> | "true" | "false" | <String> | "$" <Ident> | "$" <Ident> "." <Ident>;

Dove <Ident>, <String> ed <Integer> sono Tocken predefiniti di BNFC.

Il parser generato da Bison a partire da tale grammatica non presenta alcun tipo di conflitto.

Di seguito sono riportate tutte le modifiche effettuate al lexer ed al parser generati per implementare quanto specificato nella consegna. Ogni sezione tratta una specifica richiesta, indipendentemente dal fatto che questa sia stata realizzata a livello di lexer o parser.

Versioni

Di seguito le versioni / standard degli strumenti utilizzati per realizzare questo progetto:

1. `C` : standard `C11` ;
2. `BNFC` : 2.9.5;
3. `Flex` : 2.6.4-8.
4. `Bison` : 2:3.7.5+dfsg-1;

È fornito un `Dockerfile` con le versioni usate per semplificare l'uso del progetto.

2. Import

Il non terminale `<Import>` in BNFC ha un'unica regola:

```
ImportFile.     Import ::= String ;
```

dove `String` è il token predefinito di BNFC che riconosce le stringhe delimitate da apici. Il lexer prodotto da BNFC è stato poi modificato e reso conforme con quanto richiesto dalla consegna: gli apici sono stati sostituiti dalle le espressioni regolari `<{IMP}*import{IMP}*>` (in apertura) e `<\/{IMP}*import{IMP}*>` (in chiusura), dove `{IMP}` è un qualsiasi whitespace e/o commento; sono stati anche creati nuovi stati `IMPORT` e `ESCAPEDI` in maniera analoga a quanto viene generato per `String`.

2.a. Path

Il controllo della correttezza della stringa rappresenate il path, e quindi anche l'import del file, è delegata al parser. In particolare la funzione `make_ImportFile` generata da BNFC è stata modificata: anzichè creare un'istanza della struttura `Import`, chiama ricorsivamente la funzione `pXML` sul file da importare. Tale funzione è pensata per essere usata esclusivamente con path UNIX-like.

In questo modo l'effetto è che nell'abstract syntax tree non ci saranno mai `Import`. Per questo motivo le strutture `Import` e `XML` (la versione di `XMLS` che è preceduta da uno o più `Import`) sono state eliminate.

I path dei file da importare possono essere di due tipi:

1. path assoluti: in questo caso non c'è ambiguità, viene importato il file corrispondente se esiste, altrimenti viene ritornato un errore e viene fermato il parsing;
2. path relativi: vengono «risolti» rispetto al path del file da cui sono stati importati (o rispetto a dove è stato chiamato l'eseguibile `build` se importati dallo standard input) tramite l'apposita funzione `relative_path`.

2.b. Import ricorsivi e multipli

Per gestire import ricorsivi e multipli è stata creato un array globale `parsed_files` che viene popolato ad ogni import con il nome del file importato ed un puntatore al file che lo ha importato.

Ad esempio, consideriamo 4 file che i cui import sono descritti dal seguente grafo delle dipendenze:

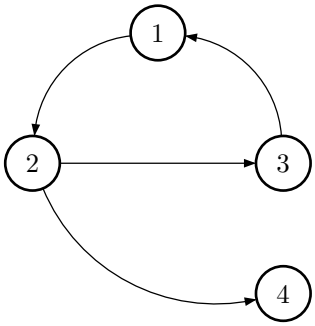


Figura 1: Esempio grafo dipendenze di import.

alla fine dell'elaborazione degli import di 2 in 1 e di 3 in 2, l'array `parsed_files` sarebbe diventato:

$$\begin{pmatrix} 1 & 2 & 3 \\ 3 & 1 & 2 \end{pmatrix} \tag{2}$$

dove la prima riga indica i nomi dei file, mentre la seconda riga i puntatori ai file che hanno importato il file nella stessa colonna (ad esempio il file 3 è importato dal file 2).

L'import successivo è nuovamente quello di 1. Prima dell'aggiunta a `parsed_files`, la funzione atta al parsing `pXML`, si sarebbe accorta della catena dei riferimenti $3 \to 2 \to 1 \to 3$, dove il 3 è ripetuto. Quindi avrebbe rifiutato l'import e stampato un opportuno warning conenente la catena di riferimenti ciclica che ha causato il problema.

Invece per verificare che non ci siano input ripetuti in generale viene controllato che non ci siano file ripetuti nella lista `parsed_files`. Anche in questo caso viene prodotto il corrispondente warning, e l'import ripetuto viene ignorato.

Infine si sarebbe continuato con l'import di 4.

3. DataStructure

La struttura dati principale è una lista concatenata di sezioni, una sezione contiene:

- il suo identificatore;
- un array dinamico di inherits (ognuno con identificatore e posizione);
- un puntatore ad una lista concatenata di field;
- il puntatore alla sezione successiva.

Un field contiene:

- il suo identificatore;
- il suo valore;
- un puntatore alla sezione a cui appartiene;
- il puntatore ad il prossimo elemento della lista.

Inoltre in ogni field sono stati aggiunti i campi `kind` e `pointed_by` per gestire i riferimenti e le eliminazioni. Un valore è una union dei vari tipi di valore possibili, vengono distinti le reference risolte e non risolte.

Tutte le strutture riportate hanno anche un capo `symbol_loc` che tiene traccia della posizione del simbolo nel file di input, in modo da poter stampare errori e warning con la riga corretta e per poter stampare i commenti associati.

Per ogni sezione viene salvato anche l'array di inherit in modo da poter controllare se ci sono inherit inutilizzati e per poter assegnare i commenti correttamente anche agli inherits.

3.a. Creazione della struttura dati

La struttura dati viene creata a partire dal AST generato dalla funzione `pXML`. Viene visitato l'albero, che in questo caso è una lista per come è stata definita la grammatica, e quindi creata la struttura. Durante questa visita vengono anche eseguiti i seguenti controlli:

- Viene verificato che non ci siano sezioni duplicate, in tal caso viene stampato un errore e il parsing viene interrotto.
- Viene verificato che non ci siano field duplicati all'interno della stessa sezione, in tal caso viene stampato un warning e il field viene sovrascritto. E' stato assunto che il valore di un field ripetuto sia l'ultimo valore assegnato.
- Viene verificato che se sono presenti variabili non locali deve esserci un inherit per la sezione che le contiene, altrimenti viene stampato un errore e il parsing viene interrotto.

3.b. Risoluzione riferimenti

Trattandosi del parsing di un file di configurazione, si è ritenuto ragionevole permettere che alcune variabili siano dichiarate anche dopo il loro utilizzo (comportamento simile all'*hoisting* in alcuni linguaggi di programmazione).

I riferimenti vengono quindi risolti alla fine del parsing dalle funzioni del modulo `ResolveRefs` direttamente sulla struttura dati `DataStructure` costruita. Nello specifico la funzione `resolveSections` scorre ricorsivamente tutte le `Sections` e itera sui loro `Fields`. Per ogni valore del tipo `Ref_unresolved` (overo una reference ancora non risolta), viene chiamata la funzione `resolveField`. `resolveField` esegue una visita simil DFS scoprendo tutta la catena di reference (sul field su cui è stata chiamata) fino ad un valore già risolto.

Sempre in stile DFS, i field vengono marcati come `Visited` o `Not_visited` per scoprire eventuali dipendenze cicliche. In tal caso viene stampato un errore e viene abortito il parsing.

Anche nel caso di reference a sezioni / field non esistenti il parsing viene fermato con un errore.

3.c. Metodi

3.c.I. Risoluzione del valore di un field

Per risalire al valore di un field si può utilizzare la funzione `get_value`, che, dopo aver trovato il field richiesto, se il valore è una reference risale la catena di riferimenti fino a trovare il valore effettivo. Non si può incorrere in cicli o riferimenti non risolti in quanto la funzione viene chiamata dopo la risoluzione dei puntatori.

3.c.II. Aggiunta

Viene aggiunto un field nella sezione specificata, se il field esiste già viene sovrascritto il valore, altrimenti viene creato un nuovo field. Se viene inserita una reference il riferimento viene risolto e viene aggiunto anche l'inherit corrispondente se non è già presente.

3.c.III. Cancellazione

La cancellazione di un field fa uso dell'array dinamico `pointed_by` che contiene tutti i field che puntano al field da eliminare. Quando il field viene cancellato, vengono anche aggiornati tutti i valori dei field presenti in `pointed_by` con il valore del field eliminato. Se il field eliminato è una reference, viene aggiornato anche il campo `pointed_by` del field puntato e vengono aggiunti eventuali inherits mancanti.

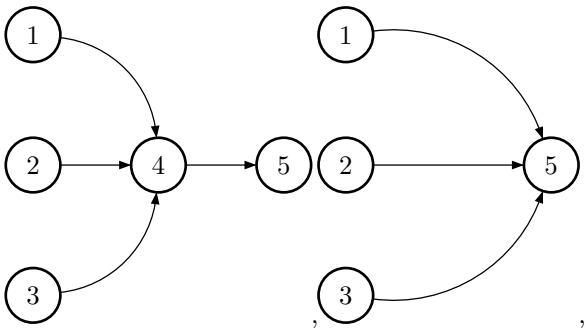


Figura 2: Esempio aggiornamento di `pointed_by` durante una cancellazione.

4. Pretty Printer

Questa sezione descrive il funzionamento del Pretty Printer progettato per il nostro linguaggio.

4.a. Struttura principale

Il printer è composto da quattro funzioni principali, ognuna responsabile della stampa di un particolare tipo di nodo nella struttura dati:

- `printSections` : stampa le sezioni del programma;
- `printInherits` : stampa gli inherits di una sezione;
- `printFields` : stampa i campi di una sezione;
- `printVal` : stampa il valore di un campo.

La funzione *printSections* prende in input una lista di sezioni e, per ciascuna sezione, stampa il nome, gli eventuali commenti associati, gli elementi ereditati (inherits) e i campi (fields). La stampa degli elementi ereditati, per ogni sezione, è gestita dalla funzione *printInherits*, che riceve in input un *DynamicArray* di elementi di tipo *Inherited* e stampa ciascun elemento insieme agli eventuali commenti associati. I campi, rappresentati da una lista di oggetti *Fields*, sono stampati tramite la funzione *printFields*, che per ogni *field* riporta il nome, i commenti eventualmente presenti e il valore. Quest'ultimo è elaborato dalla funzione *printVal*, la quale lo stampa in base al tipo (intero, stringa, booleano, riferimento), includendo anche in questo caso i commenti associati. Tutte queste funzioni fanno uso della procedura ausiliaria *get_current_comment*, che consente di ottenere il commento corrente sulla base dell'indice globale i.

4.b. Commenti

La serializzazione effettuata dal Pretty Printer deve contenere anche i commenti presenti nel file originario. A tale scopo è stato modificato il parser in modo che, invece di ignorare i commenti, li conservi nell' array globale `comments` . In particolare `comments` è un `DynamicArray` contenente elementi di tipo `Comment_` , ciascuno composto dai seguenti campi:

- `file_path` : il percorso del file in cui è presente il commento;
- `line` : il numero di riga del file in cui si trova il commento;
- `comment` : il testo del commento.

`file_path` e `line` permettono di identificare univocamente i commenti, e quindi associarli correttamente al codice in fase di print.

Quando si stampa una sezione, un `inherit`, un `field`, o un valore vengono stampati anche i commenti associati, se ve ne sono. In particolare la stampa avviene come segue:

- Per `Sections`, `inherit` e `fields`, sono considerati sia il valore `top_loc.last_line`[1] che il valore `bottom_loc.last_line`, ovvero la posizione iniziale e finale dei delimitatori dell'elemento nel file. Un commento è stampato sopra il delimitatore di un elemento se:
  1. il percorso del file del commento coincide con quello dell'elemento;
  2. la riga del commento è minore o uguale a `top_loc.last_line` (o `bottom_loc.last_line`) dell'elemento.
- Per i `Val_` viene invece utilizzato solo `bottom_loc`, poiché si assumono definiti su una singola riga. Il commento viene quindi stampato se:
  1. il percorso del file coincide;
  2. la riga del commento è minore o uguale a `bottom_loc->last_line`.

In entrambi i casi, se un commento soddisfa le condizioni, viene stampato prima dell'elemento corrispondente, e l'indice globale i viene incrementato per passare al commento successivo. Al contrario, se le condizioni non sono soddisfatte, si procede con la stampa dell'elemento corrente e si continua il controllo con l'elemento successivo.

Si nota che, nel caso degli import, il commento viene stampato all'interno della sezione del file principale, poiché il percorso del file associato al commento corrisponde a quello del file in cui è stato scritto, e non al file importato.

[1] *last_line* indica la riga in cui termina il delimitatore di un elemento. Ad esempio, nel seguente frammento:

```
1 <field name=var2> $var1
2 </field
3 >
```

`top_loc.last_line = 1`, mentre `bottom_loc->last_line = 3`.

5. Demo

I principali casi di test sono disponibili nella cartella *tests*.

È possibile verificarne il corretto funzionamento eseguendo il comando make demo da riga di comando.

ID	Descrizione	Comportamento atteso
1	Ereditarietà da una sezione precedente	Esecuzione corretta, senza errori né warning
2	Ereditarietà da una sezione successiva	Esecuzione corretta, senza errori né warning
3	Sovrascrittura di una variabile	Segnala la sovrascrittura; viene mantenuto l'ultimo valore assegnato
4	Sezione definita più volte	Segnala la sezione ripetuta; esecuzione interrotta
5	Import ricorsivo	Segnala la ricorsione sull'import; la catena viene interrotta correttamente
6	Import ripetuto dello stesso file	Segnala l'import ripetuto; viene mantenuta una sola occorrenza
7	Mancata ereditarietà per una variabile non locale	Segnala che la sezione non è ereditata; esecuzione interrotta
8	Riferimenti ricorsivi tra variabili	Segnala i riferimenti ricorsivi. Interrotta l'esecuzione
9	Riferimento a una sezione inesistente	Segnala che la sezione riferita è inesistente; esecuzione interrotta
10	Riferimento a una variabile inesistente	Segnala che la variabile riferita è inesistente; esecuzione interrotta
11	Ereditarietà superflua	Segnala l'inherit non necessario. Esegue senza modificare l'input.